

Module 4

Structured Query Language (SQL)

Overview of SQL, Data Definition Commands, Integrity constraints: key constraints, Domain Constraints, Referential integrity, check constraints, Data Manipulation commands, Data Control commands, Set and string operations, aggregate function-group by, having, Views in SQL, joins, Nested and complex queries, Triggers

Overview of SQL

SQL

- o SQL stands for Structured Query Language. It is used for storing and managing data in relational database management system (RDMS).
- o It is a standard language for Relational Database System. It enables a user to create, read, update and delete relational databases and tables.
- o All the RDBMS like MySQL, Informix, Oracle, MS Access and SQL Server use SQL as their standard database language.
- o SQL allows users to query the database in a number of ways, using English-like statements.

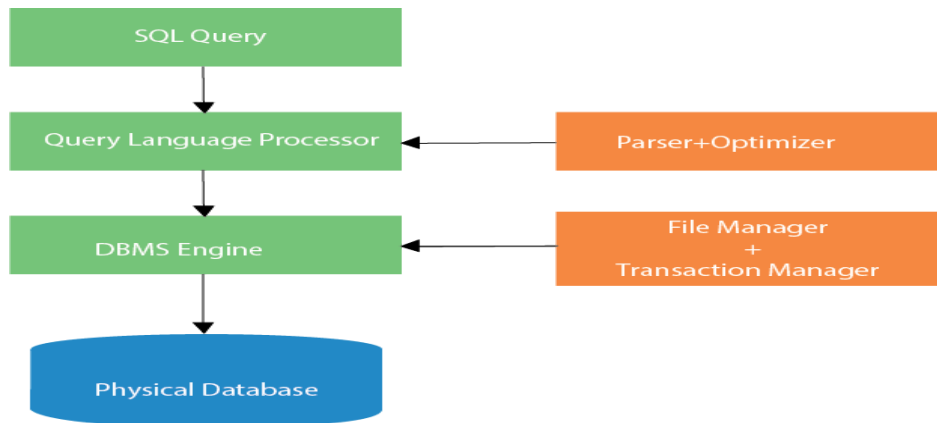
Rules:

SQL follows the following rules:

- o Structure query language is not case sensitive. Generally, keywords of SQL are written in uppercase.
- o Statements of SQL are dependent on text lines. We can use a single SQL statement on one or multiple text line.
- o Using the SQL statements, you can perform most of the actions in a database.
- o SQL depends on tuple relational calculus and relational algebra.

SQL process:

- o When an SQL command is executing for any RDBMS, then the system figure out the best way to carry out the request and the SQL engine determines that how to interpret the task.
- o In the process, various components are included. These components can be optimization Engine, Query engine, Query dispatcher, classic, etc.
- o All the non-SQL queries are handled by the classic query engine, but SQL query engine won't handle logical files.



Characteristics of SQL

- o SQL is easy to learn.
- o SQL is used to access data from relational database management systems.
- o SQL can execute queries against the database.
- o SQL is used to describe the data.
- o SQL is used to define the data in the database and manipulate it when needed.
- o SQL is used to create and drop the database and table.
- o SQL is used to create a view, stored procedure, function in a database.
- o SQL allows users to set permissions on tables, procedures, and views.

Advantages of SQL

There are the following advantages of SQL:

High speed

Using the SQL queries, the user can quickly and efficiently retrieve a large amount of records from a database.

No coding needed

In the standard SQL, it is very easy to manage the database system. It doesn't require a substantial amount of code to manage the database system.

Well defined standards

Long established are used by the SQL databases that are being used by ISO and ANSI.

Portability

SQL can be used in laptop, PCs, server and even some mobile phones.

Interactive language

SQL is a domain language used to communicate with the database. It is also used to receive answers to the complex questions in seconds.

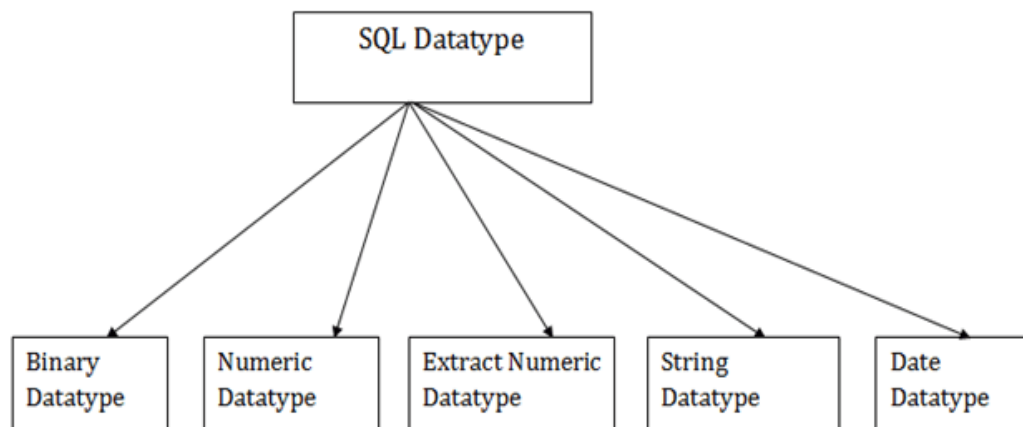
Multiple data view

Using the SQL language, the users can make different views of the database structure.

SQL Datatype

- o SQL Datatype is used to define the values that a column can contain.
- o Every column is required to have a name and data type in the database table.

Datatype of SQL:



1. Binary Datatypes

There are Three types of binary Datatypes which are given below:

Data Type	Description
binary	It has a maximum length of 8000 bytes. It contains fixed-length binary data.
varbinary	It has a maximum length of 8000 bytes. It contains variable-length binary data.

image	It has a maximum length of 2,147,483,647 bytes. It contains variable-length binary data.
-------	--

2. Approximate Numeric Datatype :

The subtypes are given below:

Data type	From	To	Description
float	-1.79E + 308	1.79E + 308	It is used to specify a floating-point value e.g. 6.2, 2.9 etc.
real	-3.40e + 38	3.40E + 38	It specifies a single precision floating point number

3. Exact Numeric Datatype

The subtypes are given below:

Data type	Description
int	It is used to specify an integer value.
smallint	It is used to specify small integer value.
bit	It has the number of bits to store.
decimal	It specifies a numeric value that can have a decimal number.
numeric	It is used to specify a numeric value.

4. Character String Datatype

The subtypes are given below:

Data type	Description
char	It has a maximum length of 8000 characters. It contains Fixed-length non-unicode characters.
varchar	It has a maximum length of 8000 characters. It contains variable-length non-unicode characters.
text	It has a maximum length of 2,147,483,647 characters. It contains variable-length non-unicode characters.

5. Date and time Datatypes

The subtypes are given below:

Datatype	Description
date	It is used to store the year, month, and days value.
time	It is used to store the hour, minute, and second values.
timestamp	It stores the year, month, day, hour, minute, and the second value.

Data Definition Commands

Data definition commands are used to create, modify and remove database objects such as schemas, tables, views, indexes etc.

Common Data Definition commands –

Create

The main use of create command is to create a new table in database. It has a predefined syntax in which we specify the columns and their respective data types.

syntax

```
CREATE TABLE <TABLE NAME>
( <COLUMN NAME> <DATA TYPE>,
<COLUMN NAME> <DATA TYPE>,
<COLUMN NAME> <DATA TYPE>,
```

```
<COLUMN NAME> <DATA TYPE>  
);
```

Example

Create a student table with columns student name and roll number.

```
CREATE TABLE STUDENT  
(STUDENT_NAME VARCHAR(30),  
ROLL_NUMBER INT  
);
```

Alter

An existing database object can be modified using the alter command. Alter command can do following changes to any table-

- Add new columns.
- Add new integrity constraints.
- Modify existing columns.
- Drop integrity constraints.

Syntax

General Syntax of the ALTER command is mentioned below –

For adding a new column

```
ALTER TABLE <table_name> ADD <column_name>
```

For renaming a table

```
ALTER TABLE <table_name> RENAME To <new_table_name >
```

For modifying a column

```
ALTER TABLE <table_name> MODIFY <column_name > <data type >
```

For deleting a column

```
ALTER TABLE <table_name> DROP COLUMN <column_name>
```

Drop

This command can delete an index, table or view. Basically, any component from a relational database management system can be removed using the Drop command. Once the object is dropped, it cannot be reused. The general syntax of drop command is as follows –

```
DROP TABLE <table_name>;  
DROP DATABASE <database_name>;  
DROP TABLE <index_name>;
```

Truncate

Using the truncate command, all the records in a database are deleted, but the database structure is maintained.

syntax

TRUNCATE TABLE <table name>

Comment

This command is used to add comments to the data dictionary.

syntax

- Single line comments: use ‘ -- ‘ before any text.
- Multiline comments: /* comments in between */

Rename

The rename command renames an object

Syntax

Rename <old name> to <new name>

Data Manipulation commands

Data manipulation commands are used to manipulate data in the database.

Some of the Data Manipulation Commands are-

Select

SELECT is the most important data manipulation command in Structured Query Language. The SELECT command shows the records of the specified table. It also shows the particular record of a particular column by using the WHERE clause.

Select statement retrieves the data from database according to the constraints specifies alongside.

Syntax of SELECT DML command

1. **SELECT** column_Name_1, column_Name_2,, column_Name_N **FROM** Name_of_table;

Here, **column_Name_1, column_Name_2,, column_Name_N** are the names of those columns whose data we want to retrieve from the table.

If we want to retrieve the data from all the columns of the table, we have to use the following SELECT command:

1. **SELECT * FROM** table_name;

Examples of SELECT Command

Example 1: This example shows all the values of every column from the table.

1. **SELECT * FROM** Student;

This SQL statement displays the following values of the student table:

Student_ID	Student_Name	Student_Marks
BCA1001	Abhay	85
BCA1002	Anuj	75
BCA1003	Bheem	60
BCA1004	Ram	79
BCA1005	Sumit	80

Example 2: This example shows all the values of a specific column from the table.

1. **SELECT** Emp_Id, Emp_Salary **FROM** Employee;
This **SELECT** statement displays all the values of **Emp_Salary** and **Emp_Id** column of **Employee** table:

Emp_Id	Emp_Salary
201	25000
202	45000
203	30000
204	29000
205	40000

Example 3: This example describes how to use the **WHERE** clause with the **SELECT DML** command.

Let's take the following Student table:

Student_ID	Student_Name	Student_Marks
------------	--------------	---------------

BCA1001	Abhay	80
BCA1002	Ankit	75
BCA1003	Bheem	80
BCA1004	Ram	79
BCA1005	Sumit	80

If you want to access all the records of those students whose marks is 80 from the above table, then you have to write the following DML command in SQL:

1. **SELECT * FROM Student WHERE Stu_Marks = 80;**
The above SQL query shows the following table in result:

Student_ID	Student_Name	Student_Marks
BCA1001	Abhay	80
BCA1003	Bheem	80
BCA1005	Sumit	80

INSERT DML Command

INSERT is another most important data manipulation command in Structured Query Language, which allows users to insert data in database tables.

Syntax of INSERT Command

1. **INSERT INTO TABLE_NAME (column_Name1 , column_Name2 , column_Name3 , column_NameN) VALUES (value_1, value_2, value_3, value_N) ;**

Examples of INSERT Command

Example 1: This example describes how to insert the record in the database table.

Let's take the following student table, which consists of only 2 records of the student.

Stu_Id	Stu_Name	Stu_Marks	Stu_Age
--------	----------	-----------	---------

101	Ramesh	92	20
201	Jatin	83	19

Suppose, you want to insert a new record into the student table. For this, you have to write the following DML INSERT command:

1. **INSERT INTO** Student (Stu_id, Stu_Name, Stu_Marks, Stu_Age) **VALUES** (104, Anmol, 89, 19);

UPDATE DML Command

UPDATE is another most important data manipulation command in Structured Query Language, which allows users to update or modify the existing data in database tables.

Syntax of UPDATE Command

1. **UPDATE** Table_name **SET** [column_name1= value_1,, column_nameN = value_N] **WHERE** CONDITION;
- Here, 'UPDATE', 'SET', and 'WHERE' are the SQL keywords, and 'Table_name' is the name of the table whose values you want to update.

Examples of the UPDATE command

Example 1: This example describes how to update the value of a single field.

Let's take a Product table consisting of the following records:

Product_Id	Product_Name	Product_Price	Product_Quantity
P101	Chips	20	20
P102	Chocolates	60	40
P103	Maggi	75	5
P201	Biscuits	80	20
P203	Namkeen	40	50

Suppose, you want to update the Product_Price of the product whose Product_Id is P102. To do this, you have to write the following DML UPDATE command:

1. **UPDATE** Product **SET** Product_Price = 80 **WHERE** Product_Id = 'P102' ;

Example 2: This example describes how to update the value of multiple fields of the database table.

Let's take a Student table consisting of the following records:

Stu_Id	Stu_Name	Stu_Marks	Stu_Age
101	Ramesh	92	20
201	Jatin	83	19
202	Anuj	85	19
203	Monty	95	21
102	Saket	65	21
103	Sumit	78	19
104	Ashish	98	20

Suppose, you want to update Stu_Marks and Stu_Age of that student whose Stu_Id is 103 and 202. To do this, you have to write the following DML Update command:

1. **UPDATE** Student **SET** Stu_Marks = 80, Stu_Age = 21 **WHERE** Stu_Id = 103 AND Stu_Id = 202;

DELETE DML Command

DELETE is a DML command which allows SQL users to remove single or multiple existing records from the database tables.

This command of Data Manipulation Language does not delete the stored data permanently from the database. We use the WHERE clause with the DELETE command to select specific rows from the table.

Syntax of DELETE Command

1. **DELETE FROM** Table_Name **WHERE** condition;

Examples of DELETE Command

Example 1: This example describes how to delete a single record from the table.

Let's take a Product table consisting of the following records:

Product_Id	Product_Name	Product_Price	Product_Quantity
P101	Chips	20	20
P102	Chocolates	60	40
P103	Maggi	75	5
P201	Biscuits	80	20
P203	Namkeen	40	50

Suppose, you want to delete that product from the Product table whose Product_Id is P203. To do this, you have to write the following DML DELETE command:

1. **DELETE FROM** Product **WHERE** Product_Id = 'P202' ;

Example 2: This example describes how to delete the multiple records or rows from the database table.

Let's take a Student table consisting of the following records:

Stu_Id	Stu_Name	Stu_Marks	Stu_Age
101	Ramesh	92	20
201	Jatin	83	19
202	Anuj	85	19
203	Monty	95	21
102	Saket	65	21
103	Sumit	78	19
104	Ashish	98	20

Suppose, you want to delete the record of those students whose Marks is greater than 70. To do this, you have to write the following DML Update command:

1. **DELETE FROM** Student **WHERE** Stu_Marks > 70 ;

Data Control commands

DCL commands are used to grant and take back authority from any database user.

Here are some commands that come under DCL:

- o Grant
- o Revoke

a. Grant: It is used to give user access privileges to a database.

Example

1. GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;

b. Revoke: It is used to take back permissions from the user.

Example

1. REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;

Transaction Control Language

TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.

These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.

Here are some commands that come under TCL:

- o COMMIT
- o ROLLBACK
- o SAVEPOINT

a. Commit: Commit command is used to save all the transactions to the database.

Syntax:

1. COMMIT;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. COMMIT;

b. Rollback: Rollback command is used to undo transactions that have not already been saved to the database.

Syntax:

1. ROLLBACK;

Example:

1. DELETE FROM CUSTOMERS
2. WHERE AGE = 25;
3. ROLLBACK;

c. SAVEPOINT: It is used to roll the transaction back to a certain point without rolling back the entire transaction.

Syntax:

1. SAVEPOINT SAVEPOINT_NAME;

Data Query Language

DQL is used to fetch the data from the database.

It uses only one command:

- o SELECT

a. SELECT: This is the same as the projection operation of relational algebra. It is used to select the attribute based on the condition described by WHERE clause.

Syntax:

1. SELECT expressions
2. FROM TABLES
3. WHERE conditions;

For example:

1. SELECT emp_name
2. FROM employee
3. WHERE age > 20;

Integrity constraints: key constraints, Domain Constraints, Referential integrity , check constraints

Integrity Constraints

- o Integrity constraints are a set of rules. It is used to maintain the quality of information.
- o Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- o Thus, integrity constraint is used to guard against accidental damage to the database.

1. Domain constraints

- o Domain constraints can be defined as the definition of a valid set of values for an attribute.
- o The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

2. Entity integrity constraints

- o The entity integrity constraint states that primary key value can't be null.
- o This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.
- o A table can contain a null value other than the primary key field.

Example:

EMPLOYEE

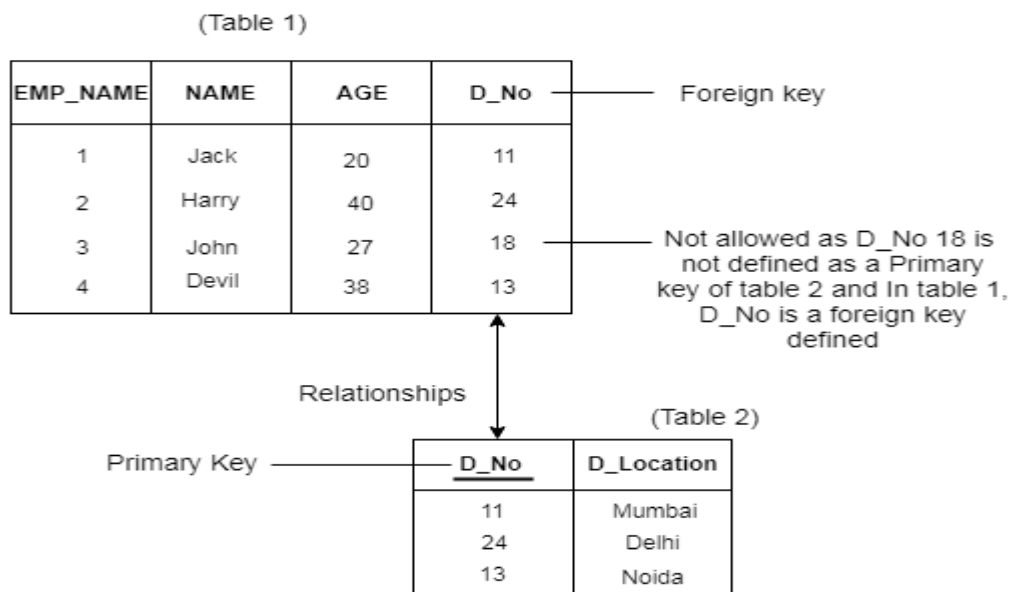
EMP_ID	EMP_NAME	SALARY
123	Jack	30000
142	Harry	60000
164	John	20000
	Jackson	27000

Not allowed as primary key can't contain a NULL value

3. Referential Integrity Constraints

- o A referential integrity constraint is specified between two tables.
- o In the Referential integrity constraints, if a foreign key in Table 1 refers to the Primary Key of Table 2, then every value of the Foreign Key in Table 1 must be null or be available in Table 2.

Example:



4. Key constraints

- o Keys are the entity set that is used to identify an entity within its entity set uniquely.
- o An entity set can have multiple keys, but out of which one key will be the primary key. A primary key can contain a unique and null value in the relational table.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1002	Morgan	8 th	22

Not allowed. Because all row must be unique

Check Constraint in SQL

- o Check constraint is validation or a rule which is applied on the column of a table.
- o When we apply check constraint to any column, it checks for specific values while inserting any record.
- o Check constraint can be created on a table level as well as column level.
- o Check constraints can be applied only to a single column, but there can be multiple check constraints in a single table.

Set and string operations

SQL Set Operation

The SQL Set operation is used to combine the two or more SQL SELECT statements.

Types of Set Operation

1. Union
2. UnionAll
3. Intersect
4. Minus

1. Union

- o The SQL Union operation is used to combine the result of two or more SQL SELECT queries.

- o In the union operation, all the number of datatype and columns must be same in both the tables on which UNION operation is being applied.
- o The union operation eliminates the duplicate rows from its resultset.

Syntax

1. SELECT column_name FROM table1
2. UNION
3. SELECT column_name FROM table2;

Example:

The First table

ID	NAME
1	Jack
2	Harry
3	Jackson

The Second table

ID	NAME
3	Jackson
4	Stephan
5	David

Union SQL query will be:

1. SELECT * FROM First
2. UNION
3. SELECT * FROM Second;

The resultset table will look like:

ID	NAME
1	Jack

2	Harry
3	Jackson
4	Stephan
5	David

2. Union All

Union All operation is equal to the Union operation. It returns the set without removing duplication and sorting the data.

Syntax:

1. SELECT column_name FROM table1
2. UNION ALL
3. SELECT column_name FROM table2;

Example: Using the above First and Second table.

Union All query will be like:

1. SELECT * FROM First
2. UNION ALL
3. SELECT * FROM Second;

The resultset table will look like:

ID	NAME
1	Jack
2	Harry
3	Jackson
3	Jackson
4	Stephan
5	David

3. Intersect

- o It is used to combine two SELECT statements. The Intersect operation returns the common rows from both the SELECT statements.
- o In the Intersect operation, the number of datatype and columns must be the same.
- o It has no duplicates and it arranges the data in ascending order by default.

Syntax

1. SELECT column_name FROM table1
2. INTERSECT
3. SELECT column_name FROM table2;

Example:

Using the above First and Second table.

Intersect query will be:

1. SELECT * FROM First
2. INTERSECT
3. SELECT * FROM Second;

The resultset table will look like:

ID	NAME
3	Jackson

4. Minus

- o It combines the result of two SELECT statements. Minus operator is used to display the rows which are present in the first query but absent in the second query.
- o It has no duplicates and data arranged in ascending order by default.

Syntax:

1. SELECT column_name FROM table1
2. MINUS
3. SELECT column_name FROM table2;

Example

Using the above First and Second table.

Minus query will be:

1. SELECT * FROM First
2. MINUS
3. SELECT * FROM Second;

The resultset table will look like:

ID	NAME
1	Jack
2	Harry

SQL String Functions

What are String Functions in SQL?

SQL String functions are the predefined functions that allow the database users for string manipulation. These functions only accept, process, and give results of the string data type.

Following are the most important string functions in Structured Query Language:

1. ASCII()
2. CHAR_LENGTH()
3. CHARACTER_LENGTH()
4. CONCAT()
5. CONCAT_WS()
6. FIND_IN_SET()
7. FORMAT()
8. INSERT()
9. INSTR()
10. LCASE()
11. LEFT()
12. LOCATE()
13. LOWER()
14. LPAD()
15. LTRIM()
16. MID()
17. POSITION()

-
18. REPEAT()
 19. REPLACE()
 20. REVERSE()
 21. RIGHT()
 22. RPAD()
 23. RTRIM()
 24. SPACE()
 25. STRCMP()
 26. SUBSTR()
 27. SUBSTRING()
 28. SUBSTRING_INDEX()
 29. UCASE()
 30. UPPER()

Let's discuss each string function in brief with the SQL table.

Now, we create a new table in SQL, which helps to understand each string function. The syntax for creating a new table in the SQL database is as follows:

```
CREATE TABLE table_name
(
  1st_Column Data Type (character_size of 1st Column),
  2nd_Column Data Type (character_size of the 2nd column ),
  3rd_Column Data Type (character_size of the 3rd column),
  ...
  Nth_Column Data Type (character_size of the Nth column)
);
```

The following CREATE statement creates the **Faculty_Info** table:

```
CREATE TABLE Faculty_Info
(
  Faculty_ID INT NOT NULL PRIMARY KEY,
  Faculty_First_Name VARCHAR (100),
  Faculty_Last_Name VARCHAR (100),
  Faculty_Dept_Id INT NOT NULL,
  Faculty_AddressVarchar(120),
  Faculty_City Varchar (80),
  Faculty_Salary INT
);
```

The following INSERT queries insert the records of college Faculties in the **Faculty_Info** table:

```
INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) VALUES (1001, Arush, Sharma, 4001, Aman Vihar, Delhi, 20000);
```

INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) **VALUES** (1002, Bulbul, Roy, 4002, Nirman Vihar, Delhi, 38000);

INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) **VALUES** (1004, Saurabh, Sharma, 4001, Sector 128, Mumbai, 45000);

INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) **VALUES** (1005, Shivani, Singhania, 4001, Vivek Vihar, Kolkata, 42000);

INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) **VALUES** (1006, Avinash, Sharma, 4002, Sarvodya Colony, Delhi, 28000);

INSERT INTO Faculty_Info (Faculty_ID, Faculty_First_Name, Faculty_Last_NameFaculty_Dept_Id, Faculty_Address, Faculty_City, Faculty_Salary) **VALUES** (1007, Shyam, Besas, 4003, Krishna Nagar, Lucknow, 35000);

The following **SELECT** statement displays the inserted records of the above **Faculty_Info** table:

SELECT * FROM Faculty_Info;

Faculty_Id	Faculty_First_Name	Faculty_Last_Name
1001	Arush	Sharma
1002	Bulbul	Roy
1004	Saurabh	Roy
1005	Shivani	Singhania
1006	Avinash	Sharma
1007	Shyam	Besas

ASCII String Function

This function in SQL returns the ASCII value of the character in the output. It gives the ASCII value of the left-most character of the string.

Syntax of ASCII String Function:

Syntax1: This syntax uses ASCII with the table column:

SELECT ASCII(Column_Name) **as** ASCII_Name **FROM** Table_Name;

Syntax2: This syntax uses ASCII with the string:

SELECT ASCII(String);

Syntax3: This syntax uses ASCII with the character:

SELECT ASCII(Character);

Example of ASCII String function:

The following SELECT query uses ASCII code with the Faculty_City column of the above Faculty_Info table.

SELECT Faculty_City, ASCII(Faculty_City) **AS** ASCII_code_of_column **FROM** Faculty_Info;

This query shows the ASCII code of the first character of all cities of the Faculty_City column.

Faculty_City	ASCII_Code_of_column
Delhi	68
Delhi	68
Mumbai	77
Kolkata	75
Delhi	68
Lucknow	76

CHAR_LENGTH String Function

This string function returns the length of the specified word. It shows the number of characters from the word.

Syntax of CHAR_LENGTH String Function:

Syntax1: This syntax uses CHAR_LENGTH() with the table column:

SELECT CHAR_LENGTH(Column_Name) **as** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses CHAR_LENGTH() with the word:

```
SELECT CHAR_LENGTH(word);
```

Examples of CHAR_LENGTH String function:

Example 1: This example shows the number of characters of the JavaTpoint word:

```
SELECT CHAR_LENGTH('JavaTpoint');
```

Output:

10

Example 2: This example uses CHAR_LENGTH() with the Faculty_Last_Name column of the above Faculty_Info table.

```
SELECT Faculty_Last_Name, CHAR_LENGTH(Faculty_Last_Name) AS Length_of_Last_Namecolumn FROM Faculty_Info;
```

This query shows the total number of characters of the last name of each faculty.

Output:

Faculty_Last_Name	Length_of_Last_Namecolumn
Sharma	6
Roy	3
Roy	3
Singhania	9
Sharma	6
Besas	5

CHARACTER_LENGTH String Function

This string function returns the length of the given string. It shows the number of all characters and spaces from the sentence.

Syntax of CHARACTER_LENGTH String Function:

Syntax1: This syntax uses CHARACTER_LENGTH() with the table column:

SELECT CHARACTER_LENGTH(Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses CHARACTER_LENGTH() with the string:

SELECT CHARACTER_LENGTH(String);

Examples of CHARACTER_LENGTH String function:

Example 1: The following SELECT query shows the total number of characters and spaces of the specified string:

SELECT CHARACTER_LENGTH('JavaTpoint is a good company');

Output:

28

Example 2: The following SELECT query uses CHARACTER_LENGTH() with the Faculty_Address column of the above Faculty_Info table.

SELECT Faculty_Address, CHARACTER_LENGTH(Faculty_Address) **AS** Length_of_Address_column **FROM** Faculty_Info;

This SQL statement shows the total number of characters and spaces of the address of each faculty.

Output:

Faculty_Address	Length_of_Address_column
Aman Vihar	10
Nirman Vihar	12
Sector 128	10
Vivek Vihar	11
Sarvodya Calony	15
Krishna Nagar	13

CONCAT String Function

This string function concatenates two strings or words and forms a new string in the result.

Syntax of CONCAT String Function:

Syntax1: This syntax uses CONCAT() with table columns:

```
SELECT CONCAT(Column_Name1, Column_Name2, ..... column_NameN) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses CONCAT() with multiple strings:

```
SELECT CONCAT(String_1, String_2, String_3, ....., String_N);
```

Examples of CONCAT string function:

Example 1: The following SELECT query appends the multiple strings into a single string:

```
SELECT CONCAT('JavaTpoint', ' is', ' a', ' good', ' company.');
```

Output:

```
JavaTpoint is a good company
```

Example 2: The following SELECT query uses CONCAT() with the Faculty_First_Name and Faculty_Last_Name columns of above Faculty_Info table:

```
SELECT Faculty_First_Name, Faculty_Last_Name  
;
```

This SQL statement merges the first name and last name of each faculty as shown in the below table:

Output:

Faculty_First_Name	Faculty_Last_Name	Append_First_LastName
Arush	Sharma	Arush Sharma
Bulbul	Roy	Bulbul Roy
Saurabh	Roy	Saurabh Roy
Shivani	Singhania	Shivani Singhania
Avinash	Sharma	Avinash Sharma

Shyam	Besas	Shyam Besas
-------	-------	-------------

CONCAT_WS String Function

This string function concatenates multiple strings or words with the help of concatenating symbol. This function uses another parameter that denotes the concatenate symbol.

Syntax of CONCAT_WS String Function:

Syntax1: This syntax uses CONCAT_WS() with table columns:

SELECT CONCAT_WS(Concatenate_symbol, Column_Name1, Column_Name2, column_Name N) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses CONCAT_WS() with multiple strings:

SELECT CONCAT_WS(Concatenate_symbol, String_1, String_2, String_3,, String_N);

Examples of CONCAT_WS String function:

Example 1: The following SELECT query appends the multiple strings using the plus (+) symbol:

SELECT CONCAT_WS('+', 'JavaTpoint', ' is', ' a', ' good', ' company');

Output:

```
JavaTpoint+is+a+good+company
```

Example 2: The following SELECT query uses CONCAT_WS() with the Faculty_First_Name and Faculty_Last_Name columns of the above Faculty_Info table:

SELECT Faculty_First_Name, Faculty_Last_Name CONCAT_WS('.', Faculty_First_Name, Faculty_Last_Name) **AS** Append_First_LastName **FROM** Faculty_Info;

This SQL statement merges the first name and last name of each faculty by the dot symbol.

Output:

Faculty_First_Name	Faculty_Last_Name	Append_First_LastName
Arush	Sharma	Arush.Sharma
Bulbul	Roy	Bulbul.Roy
Saurabh	Roy	Saurabh.Roy
Shivani	Singhania	Shivani.Singhania

Avinash	Sharma	Avinash.Sharma
Shyam	Besas	Shyam.Besas

FIND_IN_SET String Function

This string function allows you to find the position of the searched_string in the set of strings.

Syntax of FIND_IN_SET String Function:

```
SELECT FIND_IN_SET(Concatenate_symbol, String_1, String_2, String_3, ....., String_N);
```

Examples of FIND_IN_SET String function:

Example 1: The following SELECT query searches 'a' character from the given set of characters:

```
SELECT FIND_IN_SET('a', 'JavaTpoint, is, a, good, company');
```

Output:

```
3
```

Example2: The following SELECT query searches 'Delhi' string from the given set of strings:

```
SELECT FIND_IN_SET('Delhi', 'Mumbai, Goa, Bangalore, Delhi, Kolkata, Chennai');
```

Output:

```
4
```

FORMAT String Function

This String function allows you to display the given string in the specified format.

Syntax of FORMAT String Function:

Syntax1: This syntax uses FORMAT() with table column:

```
SELECT FORMAT(Column_Name1, Format_String) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses FORMAT() with the string:

```
SELECT FORMAT(String_1, Format_String);
```

Examples of FORMAT String function:

Example 1: The following SELECT query displays the number in the percentage format:

```
SELECT FORMAT('0.958', 'Percent');
```

Output:

95.80%

Example 2: The following SELECT query uses FORMAT() with the Faculty_Salary column of the above Faculty_Info table:

SELECT Faculty_Salary, FORMAT(Faculty_Salary, 'C') **AS** Currency_Salary **FROM** Faculty_Info;
This SQL statement displays the salary of each faculty in the currency format.

Output:

Faculty_Salary	Currency_Salary
20000	\$20000.00
38000	\$38000.00
45000	\$45000.00
42000	\$42000.00
28000	\$28000.00
35000	\$35000.00

INSERT String Function

This string function allows the database users to insert the sub-string in the original string at the given index position.

Syntax of INSERT String Function:

Syntax1: This syntax uses INSERT() with the column of the SQL:

SELECT INSERT(Column_Name, Position, Number, String) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses INSERT() with the string:

SELECT INSERT(String_1, Position, Number, String_2);

Examples of INSERT String function:

Example 1: The following SELECT query inserts the 'Tpoint' string at the fifth position in the 'JavaExcel' string:

SELECT INSERT('JavaExcel', 5, 6, 'Tpoint');

Output:

JavaTpointExcel

Example 2: The following SELECT query uses INSERT() with the Faculty_City column of the above Faculty_Info table:

SELECT Faculty_City, **INSERT**(Faculty_City, 3, 4, 'Agra') **AS** Insert_Agra **FROM** Faculty_Info;
This SQL statement inserts the Agra string at the third position in the city of each faculty.

Output:

Faculty_City	Insert_Agra
Delhi	DeAgralhi
Delhi	DeAgralhi
Mumbai	MuAgrambai
Kolkata	KoAgralkata
Delhi	DeAgralhi
Lucknow	LuAgracknow

INSTR String Function

This string function returns the index value of the first occurrence of the given character in the string.

Syntax of INSTR String Function:

Syntax1: This syntax uses INSTR() with the column of the SQL:

SELECT INSTR(Column_Name, **character**) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses INSTR() with the string:

SELECT INSTR(String, **character**);

Examples of INSTR String function:

Example 1: The following SELECT query shows the index value of the 'T' character in the JavaTpoint string

```
SELECT INSTR('JavaTpoint', 'T');
```

Output:

```
5
```

Example 2: The following SELECT query uses INSTR() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, INSTR(Faculty_Address, 'a') AS INSTR_Address FROM Faculty_Info;
```

This SQL statement converts the cities of all faculties into lower case letters.

Output:

Faculty_Address	LCASE_Address
Aman Vihar	3
Nirman Vihar	5
Sactor 128	2
Vivek Vihar	10
Sarvodya Calony	2
Krishna Nagar	7

LCASE String Function

This string function allows users to convert the specified string into lower case letters.

Syntax of LCASE String Function:

Syntax1: This syntax uses LCASE() with the column of the SQL table:

```
SELECT LCASE(Column_Name) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses LCASE() with the string:

```
SELECT LCASE(String);
```

Examples of LCASE String function:

Example 1: The following SELECT query converts the upper case letters of the given string into the lower case letters.

```
SELECT LCASE( 'The CAPITAL of INDIA is NEW DELHI');
```

Output:

```
the capital of india is new delhi
```

Example 2: The following SELECT query uses LCASE() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, LCASE(Faculty_Address) AS LCASE_Address FROM Faculty_Info;
```

This SQL statement converts the cities of all faculties into lower case letters.

Output:

Faculty_Address	LCASE_Address
Aman Vihar	aman vihar
Nirman Vihar	nirman vihar
Sector 128	sector 128
Vivek Vihar	vivek vihar
Sarvodya Calony	sarvodya colony
Krishna Nagar	krishna nagar

LEFT String Function

This string function shows the leftmost characters from the given string. It reads the characters to the given index position.

Syntax of LEFT String Function:

Syntax1: This syntax uses LEFT() with the column of the SQL table:

```
SELECT LEFT(Column_Name, Index_position) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses LEFT() with the string:

```
SELECT LEFT(String, Index_position);
```

Examples of LEFT String function:

Example 1: The following SELECT query shows the 11 leftmost characters from the given string:

SELECT LEFT('The CAPITAL of INDIA is NEW DELHI', 11);

Output:

The CAPITAL

Example 2: The following SELECT query uses LEFT() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, LEFT(Faculty_Address, 6) **AS** LEFT_Address **FROM** Faculty_Info;

This SQL statement shows the 6 leftmost characters from the address of all faculties

Output:

Faculty_Address	LEFT_Address
Aman Vihar	Aman V
Nirman Vihar	Nirman
Sector 128	Sector
Vivek Vihar	Vivek
Sarvodya Calony	Sarvod
Krishna Nagar	Krishn

LOCATE String Function

This string function shows the index value of the first occurrence of the word in the given string.

Syntax of LOCATE String Function:

Syntax1: This syntax uses LOCATE() with the column of the SQL table:

SELECT LOCATE(Search_string, Column_Name, Search_position) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses LOCATE() with the string:

SELECT LOCATE(Search_string, String Search_position);

Examples of LOCATE String function:

Example 1: The following SELECT query shows the index value of the INDIA word in the given sentence:

```
SELECT LOCATE('INDIA','The CAPITAL of INDIA is NEW DELHI ', 1);
```

Output:

```
16
```

Example 2: The following SELECT query uses LOCATE() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, LOCATE('r ', Faculty_Address, 1) AS LOCATE_r_Address FROM Faculty_Info;
```

This SQL statement shows the index value of 'r' in the address of each faculty.

Output:

Faculty_Address	LOCATE_r_Address
Aman Vihar	10
Nirman Vihar	3
Sector 128	6
Vivek Vihar	11
Sarvodya Calony	3
Krishna Nagar	2

LOWER String Function

Learn more

This string function allows users to convert the specified string into lower case letters. This function is also the same as the LCASE() string function.

Syntax of LOWER String Function:

Syntax1: This syntax uses LOWER() with the column of the SQL table:

SELECT LOWER(Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses LOWER() with the string:

SELECT LOWER(String);

Examples of LOWER String function:

Example 1: The following SELECT query converts the upper case letters of the given string into the lower case letters.

SELECT LOWER('NEW DELHI IS THE CAPITAL OF INDIA');

Output:

```
new delhi is the capital of india
```

Example 2: The following SELECT query uses LOWER() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, LOWER(Faculty_Address) **AS** LOWER_Address **FROM** Faculty_Info;

This SQL statement converts the cities of all faculties into lower case letters.

Output:

Faculty_Address	LOWER_Address
Aman Vihar	aman vihar
Nirman Vihar	nirman vihar
Sector 128	sector 128
Vivek Vihar	vivek vihar
Sarvodya Calony	sarvodya colony
Krishna Nagar	krishna nagar

LPAD String Function

This string function adds the given symbol to the left of the given string.

Syntax of LPAD String Function:

Syntax1: This syntax uses LPAD() with the column of the SQL table:

SELECT LPAD(Column_Name, **size**, symbol) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses LPAD() with the string:

SELECT LPAD(String, **size**, symbol);

Examples of LPAD String function:

Example 1: The following SELECT query adds the # symbol three times to the left of the NEW string:

SELECT LPAD('NEW', 6, '#');

Output:

```
###NEW
```

Example 2: The following SELECT query uses LPAD() with the Faculty_City column of the above Faculty_Info table:

SELECT Faculty_City, LPAD(Faculty_City, 10, '*') **AS** LPAD_City **FROM** Faculty_Info;

This SQL statement adds the * (asterisk) symbol five times to the left of the city of all faculties:

Output:

Faculty_City	LPAD_City
Delhi	*****Delhi
Delhi	*****Delhi
Mumbai	****Mumbai
Kolkata	***Kolkata
Delhi	*****Delhi
Lucknow	***Lucknow

LTRIM String Function

This string function cuts the given character or string from the left of the given original string. It also removes the space from the left of the specified string.

Syntax of LTRIM String Function:

Syntax1: This syntax uses LTRIM() with the column of the SQL table:

SELECT LTRIM(Column_Name, string) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses LTRIM() with the string:

SELECT LTRIM(Original_String, trimmed_string);

Examples of LTRIM String function:

Example 1: The following SELECT query trims the NEW DELHI words from the specified string:

SELECT LTRIM('NEW DELHI IS THE CAPITAL OF INDIA', 'NEW DELHI');

Output:

```
IS THE CAPITAL OF INDIA
```

Example 2: The following SELECT query trims the space from the specified string:

SELECT LTRIM(' JAVATPOINT ');

Output:

```
'JAVATPOINT'
```

Example 3: The following SELECT query trims the given character from the left of specified string:

SELECT LTRIM('#####98221545', '#');

Output:

```
98221545
```

Example 4: The following SELECT query uses LTRIM() with the Faculty_Last_Name column of above Faculty_Info table:

SELECT Faculty_Last_Name, LTRIM(Faculty_Last_Name) **AS** LTRIM_LastName **FROM** Faculty_Info;

This SQL statement trims the space from the left of the last name of all faculties:

Output:

Faculty_Last_Name	LTRIM_LastName
Sharma	Sharma
Roy	Roy

Roy	Roy
Singhania	Singhania
Sharma	Sharma
Besas	Besas

MID String Function

This string function extracts the sub-string from the given position of the original string.

Syntax of MID String Function:

Syntax1: This syntax uses MID() with the column of the SQL table:

SELECT MID(Column_Name, Starting_Position, Length) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses MID() with the string:

SELECT MID(Original_String, Starting_Position, Length);

Examples of MID String function:

Example 1: The following SELECT query shows the character from the 5th to the 10th position of the string.

SELECT MID('NEW DELHI IS THE CAPITAL OF INDIA', 5, 10);

Output:

DELHI IS T

Example 2: The following SELECT query uses MID() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, MID(Faculty_Address, 3, 8) **AS** MID_Address **FROM** Faculty_Info;

This SQL statement shows the character from the 3rd position till the 8th position of the address.

Output:

Faculty_Address	MID_Address
Aman Vihar	an Vihar

Nirman Vihar	rman Vih
Sector 128	ctor 128
Vivek Vihar	vek Viha
Sarvodya Calony	rvodya C
Krishna Nagar	ishna Na

POSITION String Function

This string function finds the position of the first occurrence of the given string in the main string.

Syntax of POSITION String Function:

Syntax1: This syntax uses POSITION() with the column of the SQL table:

SELECT POSITION(String IN Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses POSITION() with the string:

SELECT POSITION(String IN Original_String);

Examples of POSITION String function:

Example 1: The following SELECT query finds the position of the IT Company string in the original string:

SELECT POSITION('IT Company' IN'javatpoint is an indian IT company');

Output:

25

Example 2: The following SELECT query finds the position of the 'H' string in the original string:

SELECT POSITION('H' IN'HINDUSTAN');

Output:

1

Example 3: The following SELECT query uses POSITION() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, POSITION('a' IN Faculty_Address) **AS** POSITION_a_IN Address **FROM** Faculty_Info;

This SQL statement finds the position of character 'a' in the address of each faculty:

Output:

Faculty_Address	POSITION_a_IN Address
Aman Vihar	3
Nirman Vihar	5
Sector 128	0
Vivek Vihar	10
Sarvodya Calony	2
Krishna Nagar	7

REPEAT String Function

This string function writes the given string or character till the given number of times.

Syntax of REPEAT String Function:

Syntax1: This syntax uses REPEAT() with the column of the SQL table:

SELECT REPEAT(Column_Name, Repetation_Number) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses REPEAT() with the string:

SELECT REPEAT(String, Repetation_Number);

Examples of REPEAT String function:

Example 1: The following SELECT query writes the given string three times in the output.

SELECT REPEAT('javatpoint is an indian IT company', 3);

Output:

```
javatpoint is an indian IT companyjavatpoint is an indian IT companyjavatpoint is an indian IT company
```

Example 2: The following SELECT query writes the given character five times in the output.

SELECT REPEAT('H ' , 5);

Output:

#####

Example 3: The following SELECT query uses REPEAT() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, REPEAT(Faculty_Address, 2) **AS** REPEAT_Address **FROM** Faculty_Info;

This SQL statement writes the address of each faculty two times in the Repeat_Address column.

Output:

Faculty_Address	REPEAT_Address
Aman Vihar	Aman ViharAman Vihar
Nirman Vihar	Nirman ViharNirman Vihar
Sector 128	Sector 128Sector 128
Vivek Vihar	Vivek ViharVivek Vihar
Sarvodya Calony	Sarvodya CalonySarvodya Calony
Krishna Nagar	Krishna NagarKrishna Nagar

REPLACE String Function

This string function cuts the given string by removing the given sub-string.

Syntax of REPLACE String Function:

Syntax1: This syntax uses REPLACE() with the column of the SQL table:

SELECT REPLACE(Column_Name, sub_string) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses REPLACE() with the string:

SELECT REPLACE(Original_String, sub_string);

Examples of REPLACE String function:

Example 1: The following SELECT query removes the 'javatpoint' word from the original string:

```
SELECT REPLACE( 'javatpoint Indian IT company javatpoint', 'javatpoint');
```

Output:

```
Indian IT company
```

Example 2: The following SELECT query removes the given character H from the string:

```
SELECT REPLACE( 'HIJHKHJKL' , 'H');
```

Output:

```
IJKJKL
```

Example 3: The following SELECT query uses REPLACE() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, REPLACE( Faculty_Address, 'a' ) AS REPLACE_a_Address FROM Faculty_Info;
```

This SQL statement removes the character a from the address of each faculty:

Output:

Faculty_Address	REPLACE_a_Address
Aman Vihar	Amn Vihr
Nirman Vihar	Nirmn Vihr
Sector 128	Sector 128
Vivek Vihar	Vivek Vihr
Sarvodya Calony	Srvody Clony
Krishna Nagar	Krishn Ngr

REVERSE String Function

This string function of Structured query Language reverses all the characters of the string.

Syntax of REVERSE String Function:

Syntax1: This syntax uses REVERSE() with the column of the SQL table:

SELECT REVERSE(Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses REVERSE() with the string:

SELECT REVERSE(String);

Examples of REVERSE String function:

Example 1: The following SELECT query reverses the characters of the JavaTpoint string:

SELECT REVERSE('javatpoint');

Output:

```
tnioptavaJ
```

Example 3: The following SELECT query uses REVERSE() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, REVERSE(Faculty_Address) **AS** REVERSE_Address **FROM** Faculty_Info;

This SQL statement reverses the address of each faculty:

Output:

Faculty_Address	REVERSE_Address
Aman Vihar	rahiv nama
Nirman Vihar	rahiv namrin
Sector 128	821 rotces
Vivek Vihar	rahiv keviv
Sarvodya Calony	ynolac aydovras
Krishna Nagar	ragan anhsirk

RIGHT String Function

This string function shows the right-most characters from the given string. It reads the characters from the right side to the given index position.

Syntax of RIGHT String Function:

Syntax1: This syntax uses RIGHT() with the column of the SQL table:

SELECT RIGHT(Column_Name, Index_position) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses RIGHT() with the string:

SELECT RIGHT(String, Index_position);

Examples of RIGHT String function:

Example 1: The following SELECT query shows the 11 right-most characters from the given string:

SELECT RIGHT('The CAPITAL of INDIA is NEW DELHI', 11);

Output:

```
s NEW DELHI
```

Example 2: The following SELECT query uses RIGHT() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, RIGHT(Faculty_Address, 7) **AS** RIGHT_Address **FROM** Faculty_Info;

This SQL statement shows the 7 right-most characters from the address of each faculty.

Output:

Faculty_Address	RIGHT_Address
Aman Vihar	n Vihar
Nirman Vihar	n Vihar
Sector 128	tor 128
Vivek Vihar	k Vihar
Sarvodya Calony	Calony
Krishna Nagar	a Nagar

RPAD String Function

This string function adds the given symbol to the right of the given string.

Syntax of RPAD String Function:

Syntax1: This syntax uses RPAD() with the column of the SQL table:

SELECT RPAD(Column_Name, **size**, symbol) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses RPAD() with the string:

SELECT RPAD(String, **size**, symbol);

Examples of RPAD String function:

Example 1: The following SELECT query adds the # symbol three times to the right of the NEW string:

SELECT RPAD('NEW', 6, '#');

Output:

```
NEW###
```

Example 2: The following SELECT query uses RPAD() with the Faculty_City column of the above Faculty_Info table:

SELECT Faculty_City, RPAD(Faculty_City, 10, '*') **AS** RPAD_City **FROM** Faculty_Info;

This SQL statement adds the * (asterisk) symbol to the right of the city of each faculty.

Output:

Faculty_City	RPAD_City
Delhi	Delhi*****
Delhi	Delhi*****
Mumbai	Mumbai****
Kolkata	Kolkata***
Delhi	Delhi*****
Lucknow	Lucknow***

RTRIM String Function

This string function cuts the given character or string from the right of the given original string. It also removes the space from the right of the specified string.

Syntax of RTRIM String Function:

Syntax1: This syntax uses RTRIM() with the column of the SQL table:

SELECT RTRIM(Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses RTRIM() with the string:

SELECT RTRIM(Original_String);

Examples of RTRIM String function:

Example 1: The following SELECT query trims the NEW DELHI words from the specified string:

SELECT RTRIM('NEW DELHI IS THE CAPITAL OF INDIA', 'CAPITAL OF INDIA');

Output:

```
NEW DELHI IS THE
```

Example 2: The following SELECT query trims the space from the right of the specified string:

SELECT RTRIM(' JAVATPOINT ');

Output:

```
'          JAVATPOINT '
```

Example 3: The following SELECT query trims the given character from the right of the specified string:

SELECT RTRIM('98221545####', '#');

Output:

```
98221545
```

Example 4: The following SELECT query uses RTRIM() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, RTRIM(Faculty_Address) **AS** rtrimaddress **FROM** Faculty_Info;

This SQL statement trims the space from the right of the address of each faculty:

Output:

Faculty_Address	rtrimaddress
Aman Vihar	Aman Vihar
Nirman Vihar	Nirman Vihar

Sector 128	Sector 128
Vivek Vihar	Vivek Vihar
Sarvodya Calony	Sarvodya Calony
Krishna Nagar	Krishna Nagar

SPACE String Function

This string function adds the specified number of spaces.

Syntax of SPACE String Function:

SELECT SPACE(Number);

Example of SPACE String function:

The following SELECT query adds the 11 spaces:

SELECT SPACE(11);

Output:



STRCMP String Function

This string function compares the two specified strings with each other. This function returns 0 if both strings in SQL are similar, returns -1 if the first string is smaller than the second string, and returns 1 if the first string is bigger than the second string.

Syntax of STRCMP String Function:

Syntax1: This syntax uses STRCMP() with the columns of the SQL table:

SELECT STRCMP(Column_Name1, Column_Name2) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses STRCMP() with the two strings:

SELECT STRCMP(String1, String2);

Examples of STRCMP String function:

Example 1: The following SELECT query compares the 'INDIA' string with the 'JavaTpoint' string.

```
SELECT STRCMP( 'INDIA', 'JavaTpoint');
```

Output:

```
-1
```

Example 2: The following SELECT query compares the 'INDIA' string with the 'Point' string.

```
SELECT STRCMP( 'INDIA', 'Point');
```

Output:

```
0
```

Example 3: The following SELECT query uses STRCMP() with the Faculty_first_Name and Faculty_Last_Name columns of the above Faculty_Info table:

```
SELECT Faculty_First_Name, Faculty_Last_Name, STRCMP(Faculty_First_Name, Faculty_Last_Name) AS STRCMP_Name FROM Faculty_Info;
```

This SQL statement compares the first name and last name of each faculty.

Output:

Faculty_First_Name	Faculty_Last_Name	STRCMP_Name
Arush	Sharma	-1
Bulbul	Roy	1
Saurabh	Roy	1
Shivani	Singhania	-1
Avinash	Sharma	1
Shyam	Besas	0

SUBSTR String Function

This string function extracts the sub-string from the given position of the original string.

Syntax of SUBSTR String Function:

Syntax1: This syntax uses SUBSTR() with the column of the SQL table:

```
SELECT SUBSTR(Column_Name, Starting_Position, Length) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses SUBSTR() with the string:

```
SELECT SUBSTR(Original_String, Starting_Position, Length);
```

Examples of SUBSTR String function:

Example 1: The following SELECT query shows the character from the 5th to the 10th position of the string.

```
SELECT SUBSTR( 'NEW DELHI IS THE CAPITAL OF INDIA', 5, 10);
```

Output:

```
DELHI IS T
```

Example 2: The following SELECT query uses SUBSTR() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, SUBSTR(Faculty_Address, 3, 8 ) AS SUBSTR_Address FROM Faculty_Info;
```

This SQL statement shows the substring from the 3rd position to the 8th position of the address.

Output:

Faculty_Address	SUBSTR_Address
Aman Vihar	an Vihar
Nirman Vihar	rman Vih
Sector 128	ctor 128
Vivek Vihar	vek Viha
Sarvodya Calony	rvodya C
Krishna Nagar	ishna Na

SUBSTRING String Function

This string function shows the character of the given index value in the original string.

Syntax of SUBSTRING String Function:

Syntax1: This syntax uses SUBSTRING() with the column of the SQL table:

SELECT SUBSTRING(Column_Name, Index_Position, Starting_Position) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses SUBSTRING() with the string:

SELECT SUBSTRING(Original_String, Index_Position, Starting_Position);

Examples of SUBSTRING String function:

Example 1: The following SELECT query shows the character of the fifth position from the left side.

SELECT SUBSTRING('NEW DELHI IS THE CAPITAL OF INDIA', 5, 1);

Output:

D

Example 2: The following SELECT query uses SUBSTRING() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, SUBSTRING(Faculty_Address, 3, 1) **AS** SUBSTRING_Address **FROM** Faculty_Info;

This SQL statement shows the character of the 3rd position from the left side of the address of each faculty.

Output:

Faculty_Address	SUBSTRING_Address
Aman Vihar	a
Nirman Vihar	r
Sector 128	c
Vivek Vihar	v
Sarvodya Calony	r
Krishna Nagar	i

SUBSTRING_INDEX String Function

This string function shows the substring before the given symbol in the original string.

Syntax of SUBSTRING_INDEX String Function:

This syntax uses SUBSTRING_INDEX() with the string:

```
SELECT SUBSTRING_INDEX(Original_String, symbol, Starting_Position);
```

Example of SUBSTRING_INDEX String function:

The following SELECT query shows the substring before the @ symbol:

```
SELECT SUBSTRING_INDEX( 'NEW DELHI@IS THE CAPITAL OF INDIA', @, 1);
```

Output:

```
NEW DELHI
```

UCASE String Function

This string function allows users to convert the specified string into upper case letters or capital letters.

Syntax of UCASE String Function:

Syntax1: This syntax uses UCASE() with the column of the SQL table:

```
SELECT UCASE(Column_Name) AS Alias_Name FROM Table_Name;
```

Syntax2: This syntax uses UCASE() with the string:

```
SELECT UCASE(String);
```

Examples of UCASE String function:

Example 1: The following SELECT query converts the lower case letters of a given string into the upper case letters.

```
SELECT UCASE( 'The CAPITAL of INDIA is NEW DELHI');
```

Output:

```
THE CAPITAL OF INDIA IS NEW DELHI'
```

Example 1: The following SELECT query converts the given small letter into a capital letter:

```
SELECT UCASE( 'e');
```

Output:

```
E
```

Example 2: The following SELECT query uses UCASE() with the Faculty_Address column of the above Faculty_Info table:

```
SELECT Faculty_Address, UCASE(Faculty_Address) AS UCASE_City FROM Faculty_Info;
```

This SQL statement converts the cities of all faculties into capital letters.

Output:

Faculty_Address	UCASE_Address
Aman Vihar	AMAN VIHAR
Nirman Vihar	NIRMAN VIHAR
Sector 128	SECTOR 128
Vivek Vihar	VIVEK VIHAR
Sarvodya Calony	SARVODYA CALONY
Krishna Nagar	KRISHNA NAGAR

UPPER String Function

This string function allows users to convert the specified string into the UPPER case letters. This function is also the same as the UCASE() string function.

Syntax of UPPER String Function:

Syntax1: This syntax uses UPPER() with the column of the SQL table:

SELECT UPPER(Column_Name) **AS** Alias_Name **FROM** Table_Name;

Syntax2: This syntax uses UPPER() with the string:

SELECT UPPER(String);

Examples of UPPER String function:

Example 1: The following SELECT query converts the LOWER caseletters of the given string into the UPPER case letters.

SELECT UPPER('new delhi is the capital of India');

Output:

```
NEW DELHI IS THE CAPITAL OF INDIA
```

Example 2: The following SELECT query uses UPPER() with the Faculty_Address column of the above Faculty_Info table:

SELECT Faculty_Address, UPPER(Faculty_Address) **AS** UPPER_Address **FROM** Faculty_Info;

This SQL statement converts the cities of all faculties into the UPPER case letters.

Output:

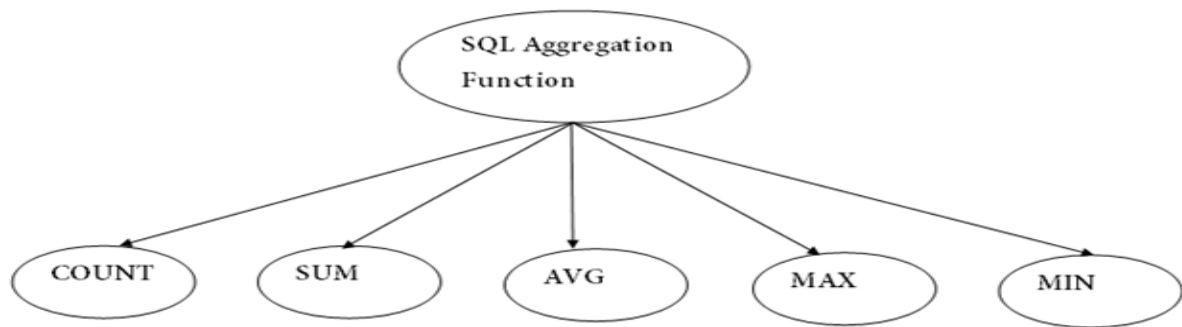
Faculty_Address	UPPER_Address
Aman Vihar	AMAN VIHAR
Nirman Vihar	NIRMAN VIHAR
Sector 128	SECTOR 128
Vivek Vihar	VIVEK VIHAR
Sarvodya Calony	SARVODYA CALONY
Krishna Nagar	KRISHNA NAGAR

Aggregate function-group by, having

SQL Aggregate Functions

- **SQL aggregation function is used to perform the calculations on multiple rows of a single column of a table. It returns a single value.**
- **It is also used to summarize the data.**

Types of SQL Aggregation Function



1. COUNT FUNCTION

- **COUNT** function is used to Count the number of rows in a database table. It can work on both numeric and non-numeric data types.
- **COUNT** function uses the **COUNT(*)** that returns the count of all the rows in a specified table. **COUNT(*)** considers duplicate and Null.

Syntax

1. **COUNT(*)**
2. or
3. **COUNT([ALL|DISTINCT] expression)**

Sample table:

PRODUCT_MAST

PRODUCT	COMPANY	QTY	RATE	COST
Item1	Com1	2	10	20
Item2	Com2	3	25	75
Item3	Com1	2	30	60
Item4	Com3	5	10	50
Item5	Com2	2	20	40

Item6	Cpm1	3	25	75
Item7	Com1	5	30	150
Item8	Com1	3	10	30
Item9	Com2	2	25	50
Item10	Com3	4	30	120

Example: COUNT()

1. **SELECT COUNT(*)**
2. **FROM PRODUCT_MAST;**

Output:

10

Example: COUNT with WHERE

1. **SELECT COUNT(*)**
2. **FROM PRODUCT_MAST;**
3. **WHERE RATE>=20;**

Output:

7

Example: COUNT() with DISTINCT

1. **SELECT COUNT(DISTINCT COMPANY)**
2. **FROM PRODUCT_MAST;**

Output:

3

Example: COUNT() with GROUP BY

1. **SELECT COMPANY, COUNT(*)**
2. **FROM PRODUCT_MAST**

3. GROUP BY COMPANY;

Output:

Com1 5
Com2 3
Com3 2

Example: COUNT() with HAVING

- 1. SELECT COMPANY, COUNT(*)**
- 2. FROM PRODUCT_MAST**
- 3. GROUP BY COMPANY**
- 4. HAVING COUNT(*)>2;**

Output:

Com1 5
Com2 3

2. SUM Function

Sum function is used to calculate the sum of all selected columns. It works on numeric fields only.

Syntax

- 1. SUM()**
- 2. or**
- 3. SUM([ALL|DISTINCT] expression)**

Example: SUM()

- 1. SELECT SUM(COST)**
- 2. FROM PRODUCT_MAST;**

Output:

670

Example: SUM() with WHERE

- 1. SELECT SUM(COST)**
- 2. FROM PRODUCT_MAST**

3. WHERE QTY>3;

Output:

320

Example: SUM() with GROUP BY

- 1. SELECT SUM(COST)**
- 2. FROM PRODUCT_MAST**
- 3. WHERE QTY>3**
- 4. GROUP BY COMPANY;**

Output:

Com1 150

Com2 170

Example: SUM() with HAVING

- 1. SELECT COMPANY, SUM(COST)**
- 2. FROM PRODUCT_MAST**
- 3. GROUP BY COMPANY**
- 4. HAVING SUM(COST)>=170;**

Output:

Com1 335

Com3 170

3. AVG function

The AVG function is used to calculate the average value of the numeric type. AVG function returns the average of all non-Null values.

Syntax

- 1. AVG()**
- 2. or**
- 3. AVG([ALL|DISTINCT] expression)**

Example:

1. **SELECT AVG(COST)**
2. **FROM PRODUCT_MAST;**

Output:

67.00

4. MAX Function

MAX function is used to find the maximum value of a certain column. This function determines the largest value of all selected values of a column.

Syntax

1. **MAX()**
2. **or**
3. **MAX([ALL|DISTINCT] expression)**

Example:

1. **SELECT MAX(RATE)**
2. **FROM PRODUCT_MAST;**

30

5. MIN Function

MIN function is used to find the minimum value of a certain column. This function determines the smallest value of all selected values of a column.

Syntax

1. **MIN()**
2. **or**
3. **MIN([ALL|DISTINCT] expression)**

Example:

1. **SELECT MIN(RATE)**
2. **FROM PRODUCT_MAST;**

Output:

What is the SQL Group by Clause?

The GROUP BY clause is a SQL command that is used to group rows that have the same values. The GROUP BY clause is used in the SELECT statement. Optionally it is used in conjunction with aggregate functions to produce summary reports from the database.

That's what it does, summarizing data from the database.

The queries that contain the GROUP BY clause are called grouped queries and only return a single row for every grouped item.

SQL GROUP BY Syntax

Now that we know what the SQL GROUP BY clause is, let's look at the syntax for a basic group by query.

```
SELECT          statements...          GROUP          BY
column_name1[,column_name2,...] [HAVING condition];
```

HERE

- “SELECT statements...” is the standard SQL SELECT command query.
- “GROUP BY *column_name1*” is the clause that performs the grouping based on *column_name1*.
- “[*column_name2*,...]” is optional; represents other column names when the grouping is done on more than one column.
- “[HAVING condition]” is optional; it is used to restrict the rows affected by the GROUP BY clause. It is similar to the WHERE clause.

Grouping using a Single Column

In order to help understand the effect of SQL Group By clause, let's execute a simple query that returns all the gender entries from the members table.

```
SELECT `gender` FROM `members` ;
```

gender

Female

Female

Male

Female

Male

Male

Male

Male

Male

Suppose we want to get the unique values for genders. We can use a following query –

```
SELECT `gender` FROM `members` GROUP BY `gender`;
```

Executing the above script in [MySQL workbench](#) against the Myflixdb gives us the following results.

gender

Female

Male

Note only two results have been returned. This is because we only have two gender types Male and Female. The GROUP BY clause in SQL grouped all the “Male” members together and returned only a single row for it. It did the same with the “Female” members.

Grouping using multiple columns

Suppose that we want to get a list of movie category_id and corresponding years in which they were released.

Let's observe the output of this simple query

```
SELECT `category_id`,`year_released` FROM `movies` ;
```

category_id	year_released
1	2011
2	2008
NULL	2008
NULL	2010
8	2007
6	2007
6	2007
8	2005
NULL	2012

7	1920
8	NULL
8	1920

The above result has many duplicates.

Let's execute the same query using group by in SQL –

```
SELECT `category_id`,`year_released` FROM `movies`
GROUP BY `category_id`,`year_released`;
```

Executing the above script in MySQL workbench against the myflixdb gives us the following results shown below.

category_id	year_released
NULL	2008
NULL	2010
NULL	2012
1	2011
2	2008
6	2007
7	1920
8	1920

8	2005
8	2007

The **GROUP BY** clause operates on both the category id and year released to identify unique rows in our above example.

If the category id is the same but the year released is different, then a row is treated as a unique one. If the category id and the year released is the same for more than one row, then it's considered a duplicate and only one row is shown.

Grouping and aggregate functions

Suppose we want total number of males and females in our database. We can use the following script shown below to do that.

```
SELECT      `gender`,COUNT(`membership_number`)      FROM
`members` GROUP BY `gender`;
```

Executing the above script in MySQL workbench against the myflixdb gives us the following results.

gender	COUNT('membership_number')
Female	3
Male	5

The results shown below are grouped by every unique gender value posted and the number of grouped rows is counted using the **COUNT** aggregate function.

Restricting query results using the HAVING clause

It's not always that we will want to perform groupings on all the data in a given table. There will be times when we will want to restrict our results to a certain given criteria. In such cases , we can use the HAVING clause

Suppose we want to know all the release years for movie category id 8. We would use the following script to achieve our results.

```
SELECT      *      FROM      `movies`      GROUP      BY  
`category_id`, `year_released` HAVING `category_id` = 8;
```

Executing the above script in MySQL workbench against the Myflixdb gives us the following results shown below.

movie_id	title	director	year_released	category_id
9	Honey mooners	John Schultz	2005	8
5	Daddy's Little Girls	NULL	2007	8

Note only movies with category id 8 have been affected by our GROUP BY clause.

HAVING Clause in SQL

The HAVING clause places the condition in the groups defined by the GROUP BY clause in the SELECT statement.

This SQL clause is implemented after the 'GROUP BY' clause in the 'SELECT' statement.

This clause is used in SQL because we cannot use the WHERE clause with the SQL aggregate functions. Both WHERE and HAVING clauses are used for filtering the records in SQL queries.

Difference between HAVING and WHERE Clause

The difference between the WHERE and HAVING clauses in the database is the most important question asked during an IT interview.

The following table shows the comparisons between these two clauses, but the main difference is that the **WHERE clause** uses condition for filtering records before any groupings are made, while HAVING clause uses condition for filtering values from a group.

HAVING	WHERE
1. The HAVING clause is used in database systems to fetch the data/values from the groups according to the given condition.	1. The WHERE clause is used in database systems to fetch the data/values from the tables according to the given condition.
2. The HAVING clause is always executed with the GROUP BY clause.	2. The WHERE clause can be executed without the GROUP BY clause.
3. The HAVING clause can include SQL aggregate functions in a query or statement.	3. We cannot use the SQL aggregate function with WHERE clause in statements.
4. We can only use SELECT statement with HAVING clause for filtering the records.	4. Whereas, we can easily use WHERE clause with UPDATE, DELETE, and SELECT statements.
5. The HAVING clause is used in SQL queries after the GROUP BY clause.	5. The WHERE clause is always used before the GROUP BY clause in SQL queries.

6. We can implements this SQL clause in column operations.	6. We can implements this SQL clause in row operations.
7. It is a post-filter.	7. It is a pre-filter.
8. It is used to filter groups.	8. It is used to filter the single record of the table.

Syntax of HAVING clause in SQL

1. **SELECT** column_Name1, column_Name2,, column_NameN
 aggregate_function_name(column_Name) **FROM** table_name **GROUP BY**
 column_Name1 **HAVING** condition;

Examples of HAVING clause in SQL

In this article, we have taken the following four different examples which will help you how to use the HAVING clause with different SQL aggregate functions:

Example 1: Let's take the following Employee table, which helps you to analyze the HAVING clause with SUM aggregate function:

Emp_Id	Emp_Name	Emp_Salary	Emp_City
201	Abhay	2000	Goa
202	Ankit	4000	Delhi
203	Bheem	8000	Jaipur
204 Ram	2000	Goa	

205	Sumit	5000	Delhi
-----	-------	------	-------

If you want to add the salary of employees for each city, you have to write the following query:

1. **SELECT SUM(Emp_Salary), Emp_City FROM Employee GROUP BY Emp_City;**

The output of the above query shows the following output:

SUM(Emp_Salary)	Emp_City
4000	Goa
9000	Delhi
8000	Jaipur

Now, suppose that you want to show those cities whose total salary of employees is more than 5000. For this case, you have to type the following query with the HAVING clause in SQL:

1. **SELECT SUM(Emp_Salary), Emp_City FROM Employee GROUP BY Emp_City HAVING SUM(Emp_Salary)>5000;**

The output of the above SQL query shows the following table in the output:

SUM(Emp_Salary)	Emp_City
9000	Delhi
8000	Jaipur

Example 2: Let's take the following Student_details table, which helps you to analyze the HAVING clause with the COUNT aggregate function:

Roll_No	Name	Marks	Age
1	Rithik	91	20
2	Kapil	60	19
3	Arun	82	17
4	Ram	92	18
5	Anuj	50	20
6	Suman	88	18
7	Sheetal	57	19
8	Anuj	64	20

Suppose, you want to count the number of students from the above table according to their age. For this, you have to write the following query:

1. **SELECT COUNT**(Roll_No), Age **FROM** Student_details **GROUP BY** Age ;

The above query will show the following output:

Count(Roll_No)	Age
3	20
2	19

1	17
2	18

Now, suppose that you want to show the age of those students whose roll number is more than and equals 2. For this case, you have to type the following query with the HAVING clause in SQL:

1. **SELECT COUNT(**Roll_No**), Age FROM Student_details GROUP BY Age HAVING COUNT(**Roll_No**) >= 2 ;**

The output of the above SQL query shows the following table in the output:

Count(Roll_No)	Age
3	20
2	19
2	18

Example 3: Let's take the following Employee table, which helps you to analyze the HAVING clause with MIN and MAX aggregate function:

Emp_ID	Name	Emp_Salary	Emp_Dept
1001	Anuj	9000	Finance
1002	Saket	4000	HR
1003	Raman	3000	Coding

1004	Renu	6000	Coding
1005	Seenu	5000	HR
1006	Mohan	10000	Marketing
1007	Anaya	4000	Coding
1008	Parul	8000	Finance

MIN Function with HAVING Clause:

If you want to show each department and the minimum salary in each department, you have to write the following query:

1. **SELECT MIN(Emp_Salary), Emp_Dept FROM Employee GROUP BY Emp_Dept;**

The output of the above query shows the following output:

MIN(Emp_Salary)	Emp_Dept
8000	Finance
4000	HR
3000	Coding
10000	Marketing

Now, suppose that you want to show only those departments whose minimum salary of employees is greater than 4000. For this case, you have to type the following query with the HAVING clause in SQL:

1. **SELECT MIN**(Emp_Salary), Emp_Dept **FROM** Employee **GROUP BY** Emp_Dept **HAVING MIN**(Emp_Salary) > 4000 ;

The above SQL query shows the following table in the output:

MIN(Emp_Salary)	Emp_Dept
8000	Finance
10000	Marketing

MAX Function with HAVING Clause:

In the above employee table, if you want to list each department and the maximum salary in each department. For this, you have to write the following query:

1. **SELECT MAX**(Emp_Salary), Emp_Dept **FROM** Employee **GROUP BY** Emp_Dept;

The above query will show the following output:

MAX(Emp_Salary)	Emp_Dept
9000	Finance
5000	HR
6000	Coding
10000	Marketing

Now, suppose that you want to show only those departments whose maximum salary of employees is less than 8000. For this case, you have to type the following query with the HAVING clause in SQL:

1. **SELECT MAX(Emp_Salary), Emp_Dept FROM Employee GROUP BY Emp_Dept HAVING MAX(Emp_Salary) < 8000 ;**

The output of the above SQL query shows the following table in the output:

MAX(Emp_Salary)	Emp_Dept
5000	HR
6000	Coding

Example 4: Let's take the following Employee_Dept table, which helps you to analyze the HAVING clause with AVG aggregate function:

Emp_ID	Name	Emp_Salary	Emp_Dept
1001	Anuj	8000	Finance
1002	Saket	4000	HR
1003	Raman	3000	Coding
1004	Renu	6000	Coding
1005	Seenu	5000	HR
1006	Mohan	10000	Marketing

1007	Anaya	4000	Coding
1008	Parul	6000	Finance

If you want to find the average salary of employees in each department, you have to write the following query:

1. **SELECT AVG**(Emp_Salary), Emp_Dept **FROM** Employee_Dept **GROUP BY** Emp_Dept;

The above query will show the following output:

AVG(Emp_Salary)	Emp_Dept
7000	Finance
4500	HR
6500	Coding
10000	Marketing

Now, suppose that you want to show those departments whose average salary is more than and equals 6500. For this case, you have to type the following query with the HAVING clause in SQL:

1. **SELECT AVG**(Emp_Salary), Emp_Dept **FROM** Employee_Dept **GROUP BY** Emp_Dept **HAVING AVG**(Emp_Salary) > 6500 ;

The above SQL query will show the following table in the output:

AVG(Emp_Salary)	Emp_Dept
-----------------	----------

7000	Finance
6500	Coding
10000	Marketing

Views in SQL

- Views in SQL are considered as a virtual table. A view also contains rows and columns.
- To create the view, we can select the fields from one or more tables present in the database.
- A view can either have specific rows based on certain condition or all the rows of a table.

Sample table:

Student_Detail

STU_ID	NAME	ADDRESS
1	Stephan	Delhi
2	Kathrin	Noida
3	David	Ghaziabad
4	Alina	Gurugram

Student_Marks

STU_ID	NAME	MARKS	AGE
1	Stephan	97	19
2	Kathrin	86	21
3	David	74	18
4	Alina	90	20
5	John	96	18

1. Creating view

A view can be created using the **CREATE VIEW** statement. We can create a view from a single table or multiple tables.

Syntax:

1. **CREATE VIEW view_name AS**
2. **SELECT column1, column2.....**
3. **FROM table_name**
4. **WHERE condition;**

2. Creating View from a single table

In this example, we create a View named **DetailsView** from the table **Student_Detail**.

Query:

1. **CREATE VIEW DetailsView AS**
2. **SELECT NAME, ADDRESS**
3. **FROM Student_Details**
4. **WHERE STU_ID < 4;**

Just like table query, we can query the view to view the data.

1. **SELECT * FROM DetailsView;**

Output:

NAME	ADDRESS
Stephan	Delhi
Kathrin	Noida
David	Ghaziabad

3. Creating View from multiple tables

View from multiple tables can be created by simply include multiple tables in the SELECT statement.

In the given example, a view is created named MarksView from two tables Student_Detail and Student_Marks.

Query:

1. CREATE VIEW MarksView AS
2. SELECT Student_Detail.NAME, Student_Detail.ADDRESS,
 Student_Marks.MARKS
3. FROM Student_Detail, Student_Mark
4. WHERE Student_Detail.NAME = Student_Marks.NAME;

To display data of View MarksView:

1. SELECT * FROM MarksView;

NAME	ADDRESS	MARKS
Stephan	Delhi	97
Kathrin	Noida	86
David	Ghaziabad	74

Alina	Gurugram	90
-------	----------	----

4. Deleting View

A view can be deleted using the Drop View statement.

Syntax

1. **DROP VIEW view_name;**

Example:

If we want to delete the View MarksView, we can do this as:

1. **DROP VIEW MarksView;**

Nested Queries in SQL

In nested queries, a query is written inside a query. The result of inner query is used in execution of outer query. We will use STUDENT, COURSE, STUDENT_COURSE tables for understanding nested queries.

STUDENT

S_ID	S_NAME	S_ADDRESS	S_PHONE	S_AGE
S1	RAM	DELHI	9455123451	18
S2	RAMESH	GURGAON	9652431543	18

S3	SUJIT	ROHTAK	9156253131	20
S4	SURESH	DELHI	9156768971	18

COURSE

C_ID	C_NAME
C1	DSA
C2	Programmin g
C3	DBMS

STUDENT_COURSE

S_ID	C_ID
-------------	-------------

S1	C1
S1	C3
S2	C1
S3	C2
S4	C2
S4	C3

There are mainly two types of nested queries:

- **Independent Nested Queries:** In independent nested queries, query execution starts from innermost query to outermost queries. The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query. Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.

IN: If we want to find out S_ID who are enrolled in C_NAME

‘DSA’ or ‘DBMS’, we can write it with the help of independent nested query and IN operator. From COURSE table, we can find out C_ID for C_NAME ‘DSA’ or DBMS’ and we can use these C_IDs for finding S_IDs from STUDENT_COURSE TABLE.

STEP 1: Finding C_ID for C_NAME =‘DSA’ or ‘DBMS’

Select C_ID from COURSE where C_NAME = ‘DSA’ or C_NAME = ‘DBMS’

STEP 2: Using C_ID of step 1 for finding S_ID

**Select S_ID from STUDENT_COURSE where C_ID IN
(SELECT C_ID from COURSE where C_NAME = ‘DSA’ or
C_NAME=‘DBMS’);**

The inner query will return a set with members C1 and C3 and outer query will return those S_IDs for which C_ID is equal to any member of set (C1 and C3 in this case). So, it will return S1, S2 and S4.

Note: If we want to find out names of STUDENTs who have either enrolled in ‘DSA’ or ‘DBMS’, it can be done as:

**Select S_NAME from STUDENT where S_ID IN
(Select S_ID from STUDENT_COURSE where C_ID IN
(SELECT C_ID from COURSE where C_NAME=‘DSA’ or
C_NAME=‘DBMS’));**

NOT IN: If we want to find out S_IDs of STUDENTs who have neither enrolled in ‘DSA’ nor in ‘DBMS’, it can be done as:

Select S_ID from STUDENT where S_ID NOT IN

```
(Select S_ID from STUDENT_COURSE where C_ID IN  
(SELECT C_ID from COURSE where C_NAME='DSA' or  
C_NAME='DBMS'));
```

The innermost query will return a set with members C1 and C3. Second inner query will return those S_IDs for which C_ID is equal to any member of set (C1 and C3 in this case) which are S1, S2 and S4. The outermost query will return those S_IDs where S_ID is not a member of set (S1, S2 and S4). So it will return S3.

- **Co-related Nested Queries:** In co-related nested queries, the output of inner query depends on the row which is being currently executed in outer query. e.g.; If we want to find out S_NAME of STUDENTs who are enrolled in C_ID 'C1', it can be done with the help of co-related nested query as:

```
Select S_NAME from STUDENT S where EXISTS  
( select * from STUDENT_COURSE SC where S.S_ID=SC.S_ID  
and SC.C_ID='C1');
```

For each row of STUDENT S, it will find the rows from STUDENT_COURSE where S.S_ID = SC.S_ID and SC.C_ID='C1'. If for a S_ID from STUDENT S, atleast a row exists in STUDENT_COURSE SC with C_ID='C1', then inner query will return true and corresponding S_ID will be returned as output.

Complex SQL Queries

[1.Query to find Second Highest Salary of Employee?\(click for explanation\)](#)

Answer:

Select distinct Salary from Employee e1 where 2=Select count(distinct Salary) from Employee e2 where e1.salary<=e2.salary;

Alternative Solution : Suggested by Ankit Srivastava

select min(salary)from(select distinct salary from emp order by salary desc)where rownum<=2;

[2.Query to find duplicate rows in table?\(click here for explanation \)](#)

Answer :

Select * from Employee a where rowid <>(select max(rowid) from Employee b where a.Employee_num=b.Employee_num);

[3.How to fetch monthly Salary of Employee if annual salary is given?\(click here for Explanation\)](#)

Answer:

Select Employee_name,Salary/12 as 'Monthly Salary' from employee;

[Click here to get information on ROW ID](#)

4.What is the Query to fetch first record from Employee table? (90% asked Complex SQL Queries Examples)

Answer:

Select * from Employee where Rownum =1;

Select * from Employee where Rowid= select min(Rowid) from Employee;

[Click here to get What is Rownum?](#)

5.What is the Query to fetch last record from the table?

Answer:

```
Select * from Employee where Rowid= select max(Rowid) from Employee;
```

A trigger is a set of SQL statements that reside in system memory with unique names. It is a specialized category of stored procedure that is called automatically when a database server event occurs. Each trigger is always associated with a table.

A trigger is called a special procedure because it cannot be called directly like a stored procedure. The key distinction between the trigger and procedure is that a trigger is called automatically when a data modification event occurs against a table. A stored procedure, on the other hand, must be invoked directly.

The following are the main characteristics that distinguish triggers from stored procedures:

- We cannot manually execute/invoked triggers.
- Triggers have no chance of receiving parameters.
- A transaction cannot be committed or rolled back inside a trigger.

Syntax of Trigger

We can create a trigger in **SQL Server** by using the CREATE TRIGGER statement as follows:

1. **CREATE TRIGGER** *schema.trigger_name*
2. **ON** *table_name*
3. **AFTER** {**INSERT**, **UPDATE**, **DELETE**}
4. [**NOT FOR REPLICATION**]
5. **AS**
6. {**SQL_Statements**}

The parameter descriptions of this syntax illustrate below:

schema: It is an optional parameter that defines which schema the new trigger belongs to.

trigger_name: It is a required parameter that defines the name for the new trigger.

table_name: It is a required parameter that defines the table name to which the trigger applies. Next to the table name, we need to write the AFTER clause where any events like INSERT, UPDATE, or DELETE could be listed.

NOT FOR REPLICATION: This option tells that **SQL** Server does not execute the trigger when data is modified as part of a replication process.

SQL_Statements: It contains one or more SQL statements that are used to perform actions in response to an event that occurs.

When we use triggers?

Triggers will be helpful when we need to execute some events automatically on certain desirable scenarios. For example, we have a constantly changing table and need to know the occurrences of changes and when these changes happen. If the primary table made any changes in such scenarios, we could create a trigger to insert the desired data into a separate table.

Example of Trigger in SQL Server

Let us understand how we can work with triggers in the SQL Server. We can do this by first creating a table named 'Employee' using the below statements:

1. **CREATE TABLE** Employee
2. (
3. **Id INT PRIMARY KEY,**
4. **Name VARCHAR(45),**
5. **Salary INT,**
6. **Gender VARCHAR(12),**
7. **DepartmentId INT**
8.)

Next, we will insert some record into this table as follows:

1. **INSERT INTO** Employee **VALUES** (1,'Steffan', 82000, 'Male', 3),
2. (2,'Amelie', 52000, 'Female', 2),
3. (3,'Antonio', 25000, 'male', 1),
4. (4,'Marco', 47000, 'Male', 2),
5. (5,'Eliana', 46000, 'Female', 3)

We can verify the insert operation by using the SELECT statement. We will get the below output:

1. **SELECT * FROM Employee;**

Id	Name	Salary	Gender	DepartmentId
1	Steffan	82000	Male	3
2	Amelie	52000	Female	2
3	Antonio	25000	male	1
4	Marco	47000	Male	2
5	Eliana	46000	Female	3
6	Peter	62000	Male	3

We will also create another table named 'Employee_Audit_Test' to automatically store transaction records of each operation, such as INSERT, UPDATE, or DELETE on the Employee table:

1. **CREATE TABLE Employee_Audit_Test**
2. **(**
3. **Id int IDENTITY,**
4. **Audit_Action text**
5. **)**

Now, we will create a trigger that stores transaction records of each insert operation on the Employee table into the Employee_Audit_Test table. Here we are going to create the insert trigger using the below statement:

1. **CREATE TRIGGER trInsertEmployee**
2. **ON Employee**
3. **FOR INSERT**
4. **AS**
5. **BEGIN**
6. **Declare @Id int**
7. **SELECT @Id = Id from inserted**
8. **INSERT INTO Employee_Audit_Test**
9. **VALUES ('New employee with Id = ' + CAST(@Id AS VARCHAR(10)) + ' is added at ' + CAST(Getdate() AS VARCHAR(22)))**
10. **END**

After creating a trigger, we will try to add the following record into the table:

1. **INSERT INTO** Employee **VALUES** (6,'Peter', 62000, 'Male', 3)

If no error is found, execute the **SELECT** statement to check the audit records. We will get the output as follows:

Id	Audit_Action
1	New employee with Id = 6 is added at Mar 24 2021 2:08PM

We are going to create another trigger to store transaction records of each delete operation on the Employee table into the Employee_Audit_Test table. We can create the delete trigger using the below statement:

1. **CREATE TRIGGER** trDeleteEmployee
2. **ON** Employee
3. **FOR DELETE**
4. **AS**
5. **BEGIN**
6. **Declare** @Id int
7. **SELECT** @Id = Id **from** deleted
8. **INSERT INTO** Employee_Audit_Test
9. **VALUES** ('An existing employee with Id = ' + **CAST**(@Id **AS** **VARCHAR**(10)) + ' is deleted at ' + **CAST**(Getdate() **AS** **VARCHAR**(22)))
- 10.**END**

After creating a trigger, we will delete a record from the Employee table:

1. **DELETE FROM** Employee **WHERE** Id = 2;

If no error is found, it gives the message as below:

Messages
(1 row affected)
(1 row affected)
Completion time: 2021-03-25T12:31:40.0681604+05:30

Finally, execute the **SELECT** statement to check the audit records:

Id	Audit_Action
1	New employee with Id = 6 is added at Mar 24 2021 2:08PM
2	An existing employee with Id = 2 is deleted at Mar 25 2021 12:26PM

In both the triggers code, you will notice these lines:

1. **SELECT** @Id = Id **from** inserted
2. **SELECT** @Id = Id **from** deleted

Here inserted and deleted are special tables used by the SQL Server. The inserted table keeps the copy of the row when you insert a new row into the actual table. And the deleted table keeps the copy of the row you have just deleted from the actual table.